Phone book

You should begin this task by downloading the phone-book-master.zip, extract the project and use the Eclispe (Import Existing Projects into Workspace) to import the project into your Eclipse workspace

The phone-book mini-project consists of three packages:

- `bcu.changeme.phonebook.model` contains the model classes `PhoneBookEntry` and `PhoneBook`, and two custom exception types.
- `bcu.changeme.phonebook.main` contains the main program class `Main`, and a custom exception type.
- `bcu.changeme.phonebook.test` contains unit tests for the `PhoneBook` model class and the `parseAndExecute` method in the main program class.

Once you have imported the mini-project, rename the `changeme` subpackage to a unique name of your choice which will identify the work as yours.

You should read the code given to you carefully to understand what it does. This code is complete; every feature is implemented already, and you should confirm that the unit tests all pass.

Your task is to refactor the `parseAndExecute` method in the `Main` class, using an interface and several new classes.

The ‘Command’ interface

In the `main` subpackage, create an interface named `Command` with a single method named `execute`.

- The `execute` method should take a `PhoneBook` as its only parameter.
- Its return type should be `void`.
- Its signature should also declare the possible exceptions `AlreadyPresentException` and `NotPresentException`.

Writing the 'Command' classes

Your overall task is to refactor this method so that instead of parsing and executing the command string, it simply parses the command and returns a `Command` object.

Before performing this refactoring, you must create several classes which implement the `Command` interface. Each of these classes will have a constructor and an `execute` method, and any fields necessary. The first example is given below:

```
1:  package bcu.changeme.phonebook.main;
2:
3:  import bcu.changeme.phonebook.model.*;
4:
5:  public class AddCommand implements Command {
6:      private final String name;
7:      private final String phoneNumber;
8:
9:      public AddCommand(String[] parts) throws InvalidCommandException {
10:         if(parts.length != 3) {
11:             throw new InvalidCommandException();
12:         }
13:         this.name = parts[1];
14:         this.phoneNumber = parts[2];
15:     }
16:
17:     @Override
18:     public void execute(PhoneBook phoneBook) throws AlreadyPresentException {
19:         phoneBook.addEntry(name, phoneNumber);
20:         System.out.println("Entry added.");
21:     }
22: }
```

Write this class in the `main` subpackage within your own mini-project, and read it carefully. The important parts of this class are:

- The class implements the `Command` interface.
- The constructor takes the array `parts` as its parameter, and checks that the array has the correct length for the `add [name] [phoneNumber]` command.
- The class has fields to store the necessary data from the `parts` array.
- The `execute` method performs the actual execution of the command.

Study the `AddCommand` class to see how it relates to the relevant part of the `parseAndExecute` method. Then, complete the remaining five `Command` classes:

- `ShowCommand`
- `UpdateCommand`
- `RemoveCommand`
- `ListCommand`
- `HelpCommand`

These classes should also be created in the `main` subpackage:

Refactoring the ‘parseAndExecute’ method

To perform the refactoring, change the `parseAndExecute` method in the `Main` class in the following ways:

- Rename the method as simply `parse`; the new method will only parse the command, not also execute it.

  > Don’t use Eclipse’s “refactor” menu for this — we only want to change the name where the method is declared. The `parseAndExecute` method is used in some unit tests, which we **don’t** want to change.

- Change the method’s signature so that it returns a `Command` object.
- Remove `AlreadyPresentException` and `NotPresentException` from the `throws` declaration.
- Rewrite each branch of the `if`/`else` structure so that it returns the appropriate type of `Command` object. You will need to pass the `parts` array to the constructors.

Completing the refactoring

You have changed and renamed the old `parseAndExecute` method, so that it now only parses the commands, and does not execute them.

To complete the refactoring, you must write a new `parseAndExecute` method. This new method should simply call the `parse` method to convert the `command` string to a `Command` object, and then call its `execute` method to execute the command.

Once you have completed your refactoring, you should run the `MainTest` unit tests to ensure that your new `parseAndExecute` method works correctly. If any tests fail, you will likely need to debug your `parse` method and/or the relevant `Command` class.

> As an extension task, write unit tests for your `parse` method. Test that it returns the correct type of `Command` object for each command. By adding getter methods to your `Command` classes, you can also test that the fields have the correct values from each command string.
